



UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE LICENȚĂ: Informatică

LUCRARE DE LICENȚĂ

**UVT Asist: asistent inteligent pentru accesul la
informațiile oficiale ale UVT**

COORDONATOR:

Asist. Drd. Theodor Radu Grumeza

ABSOLVENT:

Pavel Cătălin

TIMIȘOARA

2026

UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE LICENȚĂ: Informatică

LUCRARE DE LICENȚĂ

UVT Asist: asistent inteligent pentru accesul la
informațiile oficiale ale UVT

COORDONATOR:

Asist. Drd. Theodor Radu Grumeza

ABSOLVENT:

Pavel Cătălin

TIMIȘOARA

2026

Abstract

This bachelor thesis presents the design and implementation of **UVT Asist**, a local retrieval-augmented assistant dedicated to students of the West University of Timișoara. The application is delivered as a Chrome extension popup and is supported by a Flask backend that indexes official university and faculty pages, retrieves relevant evidence, and generates concise answers grounded in official sources.

The proposed system follows a local-index-first RAG architecture. UVT pages are crawled, cleaned, split into chunks, embedded locally with Ollama, stored in Qdrant, and searched using semantic retrieval combined with deterministic reranking. The assistant preserves the original user question, uses Ollama for linguistic interpretation of the query, validates the structured result in the backend, and exposes official source links for every answer. Final source selection is handled by retrieval, reranking and confidence scoring, not by the language model. When evidence is weak, the system avoids unsupported claims and communicates the limitation explicitly.

The thesis discusses the theoretical basis of conversational assistants, information retrieval, vector search, large language models and retrieval-augmented generation. It then maps these concepts to the implemented prototype, emphasizing transparency, local execution, source verification, and practical usability for students searching for schedules, admissions information, scholarships, regulations, contacts or other administrative resources.

Rezumat

Lucrarea prezintă proiectarea și implementarea aplicației **UVT Asist**, un asistent inteligent local, orientat către studenții Universității de Vest din Timișoara. Scopul sistemului este reducerea timpului necesar pentru găsirea informațiilor oficiale publicate pe site-ul universității și pe site-urile facultăților. Produsul final este o extensie Google Chrome, iar procesarea întrebărilor este realizată de un backend Flask care construiește contextul relevant și returnează răspunsuri scurte, însoțite de surse oficiale.

Soluția folosește o arhitectură de tip Retrieval-Augmented Generation. Paginile oficiale sunt descoperite, descărcate, curățate, împărțite în fragmente, transformate în reprezentări vectoriale cu Ollama și stocate în Qdrant. La interogare, sistemul păstrează întrebarea originală, folosește Ollama pentru interpretare lingvistică structurată și validează rezultatul în backend. Alegerea surselor rămâne deterministă, prin Qdrant, retrieval, reranking și scor de încredere. Pentru întrebările de navigare sau pentru cazurile cu dovezi insuficiente, backend-ul poate evita modelul generativ și poate folosi răspunsuri deterministe sau mesaje explicite de incertitudine.

Lucrarea tratează fundamentele teoretice ale asistenților conversaționali, ale sistemelor de regăsire a informației, ale modelelor de limbaj și ale căutării vectoriale, apoi le conectează cu deciziile tehnice din proiectul UVT Asist. Accentul este pus pe utilizarea surselor oficiale, pe transparența răspunsurilor, pe rularea locală a componentelor AI și pe limitarea riscului de generare a unor informații neverificate.

Declarație de integritate științifică

Declar că această lucrare de licență reprezintă un raport original și constituie propria mea muncă, realizată sub coordonarea Asist. Drd. Theodor Radu Grumeza. În această lucrare, metodele de cercetare, precum și procesele de colectare, analiză și interpretare a datelor, sunt prezentate corect, iar toate sursele au fost citate corespunzător acolo unde am utilizat idei și/sau materiale vizuale aparținând altor autori.

Am utilizat instrumente de inteligență artificială generativă pentru sprijin punctual în organizarea textului și ajustarea structurii documentului. Detaliile se regăsesc în Anexa A.

Cuprins

1	Introducere	7
2	Fundamente teoretice	10
2.1	Regăsirea informației și căutarea semantică	10
2.2	Modele de limbaj și RAG	11
2.3	Rulare locală, extensii browser și calitatea surselor	12
3	Analiza problemei și cerințe	14
3.1	Ecosistemul UVT și riscurile informaționale	14
3.2	Cerințe funcționale și nefuncționale	15
3.3	Criterii de acceptare	16
4	Arhitectura și implementarea sistemului	17
4.1	Arhitectura generală și structura proiectului	17
4.2	Indexare, model de date și stocare vectorială	19
4.3	Fluxul de interogare, retrieval, reranking și generare	20
5	Interfață și funcționalități	23
5.1	Extensia Chrome și experiența de utilizare	23
5.2	Backend, endpointuri, configurare și observabilitate	24
5.3	Răspunsuri, surse, feedback și degradare controlată	25
6	Validare și evaluare	27
6.1	Testare automată și scenarii manuale	27
6.2	Evaluare pe seturi de întrebări	28
6.3	Limite, amenințări la validitate, securitate și confidențialitate	34
7	Concluzii și direcții viitoare de dezvoltare	35
	Bibliografie	37
A	Utilizarea instrumentelor de inteligență artificială generativă	39
A.1	Detalii privind utilizarea OpenAI Codex / ChatGPT	39

Capitolul 1

Introducere

Digitalizarea serviciilor universitare a schimbat modul în care studenții interacționează cu instituția. Informații precum orare, calendare administrative, regulamente, metodologii de burse, proceduri de admitere, contacte, formulare sau anunțuri sunt publicate în mod curent pe site-uri web. Această disponibilitate nu înseamnă însă automat acces rapid. Studentul trebuie să știe unde să caute, să identifice pagina corectă, să interpreteze termenii folosiți de instituție și să distingă între o pagină generală, o știre veche și o metodologie încă relevantă.

În cazul Universității de Vest din Timișoara, informația este distribuită între site-ul central al universității, site-ul de admitere și site-urile facultăților. Fiecare site are propriile meniuri, propriile convenții de denumire și propriul ritm de actualizare. O întrebare aparent simplă, de exemplu „Unde găsesc orarul?” sau „Se pot cumula bursele?”, poate necesita parcurgerea mai multor pagini înainte de obținerea unei surse oficiale potrivite. Problema nu este lipsa informației, ci costul de acces la informația corectă.

Lucrarea pornește de la această observație și propune un asistent integrat în browser, capabil să răspundă la întrebări formulate natural și să indice sursele oficiale folosite. Integrarea sub forma unei extensii Chrome este potrivită pentru acest scenariu deoarece păstrează asistentul în același context în care apare nevoia de informare, iar structura extensiilor moderne este definită de mecanismele Manifest V3 [Goo]. Studentul nu este obligat să deschidă o aplicație separată, ci poate interoga sistemul direct din browser.

Asistenții conversaționali bazați pe modele de limbaj sunt utili pentru interacțiunea în limbaj natural, dar într-un context instituțional un răspuns fluent nu este suficient. Studentul are nevoie de informații verificabile, iar sistemul trebuie să evite răspunsurile speculative. În domenii precum bursele, admiterea, regulamentele sau procedurile administrative, o formulare aproximativă poate produce confuzie. Din acest motiv, UVT Asist nu tratează modelul de limbaj ca sursă unică de adevăr. Modelul este folosit pentru interpretare lingvistică și formu-

lare, iar informația factuală este extrasă din pagini oficiale selectate determinist. Abordarea este apropiată de paradigma Retrieval-Augmented Generation, în care generarea răspunsului este condiționată de context recuperat dintr-o colecție de documente [LPP⁺20, GXG⁺23].



Figura 1.1: Exemplu de context web UVT în care utilizatorul poate avea nevoie de asistență pentru navigare

Pe baza acestei probleme, scopul lucrării este proiectarea și realizarea unui prototip funcțional pentru un asistent digital care facilitează accesul la informațiile oficiale UVT. Prototipul urmărește să fie util în situații frecvente pentru studenți: găsirea orarului, identificarea datelor de contact, orientarea către informații despre admitere, consultarea paginilor despre burse, regulamente sau alte resurse administrative.

Obiectivele principale ale lucrării vizează proiectarea unei arhitecturi locale în care extensia Chrome reprezintă interfața principală, iar backend-ul Flask orchestrează căutarea și generarea răspunsului. În acest scop, proiectul urmărește construirea unui index local pornind de la pagini oficiale UVT și de la paginile facultăților, utilizarea embeddings-urilor și a unei baze vectoriale pentru regăsire semantică, precum și folosirea Ollama pentru interpretarea lingvistică structurată a întrebării, cu validare în backend și fallback pe întrebarea originală.

Un alt obiectiv important îl constituie filtrarea rezultatelor în funcție de facultate și de tipul paginii, urmată de combinarea scorurilor semantice cu reguli deterministe de reranking pentru prioritizarea surselor potrivite. Sistemul urmărește să genereze răspunsuri concise, în limba română, însoțite de sursele oficiale folosite, și să trateze explicit cazurile cu încredere scăzută, fără a inventa informații nesuținute de surse.

Contribuția principală a lucrării constă în proiectarea unei soluții aplicate, adaptate ecosistemului UVT, care combină o interfață ușor de folosit cu un mecanism local de retrieval și generare. Față de o simplă căutare pe site, sistemul acceptă întrebări naturale, folosește interpretare lingvistică pentru typo-uri și formulări imperfecte, apoi prioritizează sursele prin reguli deterministe. Față de un chatbot generic, sistemul păstrează legătura cu sursele oficiale și comunică nivelul de încredere al răspunsului.

La nivel tehnic, proiectul propune un pipeline complet: descoperirea paginilor, extragerea textului, clasificarea paginilor, segmentarea în fragmente, construirea reprezentărilor vectoriale, stocarea în Qdrant, interpretarea lingvistică a întrebării, validarea rezultatului structurat, căutarea semantică, reranking-ul determinist, calculul nivelului de încredere și generarea locală cu Ollama. Aceste componente sunt legate de direcții consacrate din regăsirea informației, embeddings, baze vectoriale și RAG [MRS08, RG19, Qdr, Oll]. La nivel de interacțiune, extensia oferă selecția facultății, istoricul conversației, întrebări recente, afișarea surselor și stări clare pentru backend indisponibil sau indexare în desfășurare.

Lucrarea este organizată în șapte capitole. Capitolul al doilea prezintă fundamentele teoretice relevante: regăsirea informației, căutarea semantică, modelele de limbaj și paradigma Retrieval-Augmented Generation. Capitolul al treilea analizează ecosistemul UVT, cerințele și criteriile de acceptare. Capitolul al patrulea descrie arhitectura și implementarea sistemului, iar capitolul al cincilea prezintă interfața și funcționalitățile aplicației. Capitolul al șaselea discută validarea, evaluarea și limitele soluției. Capitolul final sintetizează concluziile, contribuțiile, rezultatele și direcțiile viitoare.

Capitolul 2

Fundamente teoretice

2.1 Regăsirea informației și căutarea semantică

Regăsirea informației este domeniul care studiază identificarea documentelor relevante pentru o nevoie de informare exprimată printr-o interogare. În sistemele clasice, documentele sunt indexate prin termeni, iar potrivirea se face prin măsuri lexicale. Modelele de tip TF-IDF sau BM25 valorifică frecvența termenilor și raritatea lor în colecție pentru a estima relevanța [MRS08, RZ09].

Aceste metode au avantaje importante: sunt eficiente, interpretabile și nu necesită infrastructură complexă. Totuși, ele depind mult de suprapunerea exactă dintre termenii întrebării și termenii documentului. Într-un sistem destinat studenților, această dependență poate fi problematică. Utilizatorii pot întreba „unde găsesc programul?” în loc de „orar”, pot scrie cu greșeli sau pot folosi forme flexionare diferite. De aceea, sistemul are nevoie de o etapă separată de înțelegere a întrebării, completată de semnale verificabile din titlu, URL și tipul paginii.

În UVT Asist, căutarea lexicală nu este eliminată. Ea rămâne importantă pentru reranking și fallback, deoarece oferă semnale precise pentru întrebări de navigare. În schimb, pentru întrebări mai libere, precum cele despre cumularea burselor, căutarea semantică ajută la identificarea documentelor relevante chiar dacă formularea exactă diferă.

Embeddings-urile sunt reprezentări numerice dense ale textului. Ideea de bază este că texte cu sens apropiat sunt mapate în zone apropiate ale unui spațiu vectorial, iar această reprezentare permite regăsirea densă a pasajelor relevante în sisteme de întrebare-răspuns [KOM⁺20, RG19]. O întrebare și un fragment de document pot fi comparate prin similaritate cosinus:

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|},$$

unde q este vectorul întrebării, iar d este vectorul fragmentului de document. Modelele de tip Sentence-BERT ilustrează utilitatea embeddings-urilor de propoziție pentru comparații semantice eficiente [RG19].

Căutarea vectorială este utilă deoarece poate recupera documente relevante chiar și atunci când termenii nu coincid exact. Într-o colecție universitară, aceeași nevoie poate fi exprimată diferit: „taxe”, „achitare”, „plată”, „secretariat”, „program cu publicul” sau „contact”. Pentru stocarea și căutarea embeddings-urilor, proiectul folosește Qdrant, o bază de date vectorială care permite căutare după similaritate și filtrare pe metadata [Qdr]. Literatura privind căutarea la scară mare arată că organizarea eficientă a vectorilor este esențială atunci când colecțiile cresc ca volum [JDJ17].

2.2 Modele de limbaj și RAG

Modelele de limbaj moderne sunt construite pentru a prezice și genera text. Arhitectura Transformer a devenit baza multor modele actuale deoarece mecanismul de atenție permite modelarea eficientă a relațiilor dintre cuvinte pe distanțe mari [VSP⁺17]. Aceste modele pot produce răspunsuri fluente și pot înțelege într-o anumită măsură intenția utilizatorului, ceea ce le face utile pentru interfețe conversaționale.

În același timp, un model de limbaj nu garantează corectitudinea factuală a răspunsului. Dacă este întrebat despre o regulă administrativă și nu primește context oficial, modelul poate genera un răspuns plauzibil, dar greșit sau depășit. Într-o aplicație universitară, riscul este semnificativ deoarece utilizatorul poate lua decizii pe baza răspunsului.

Retrieval-Augmented Generation combină două etape: recuperarea documentelor relevante și generarea răspunsului pe baza acestora [LPP⁺20]. În loc ca modelul să răspundă doar din parametrii săi interni, sistemul îi oferă un context explicit. Această abordare este potrivită pentru domenii în care informația se schimbă sau trebuie justificată prin surse. Survey-urile recente descriu evoluția RAG de la fluxuri simple către arhitecturi modulare, cu componente separate pentru retrieval, filtrare, generare și evaluare [GXG⁺23, GYZ⁺25].

În sistemele RAG, etapa de retrieval poate fi precedată de *query understanding* sau *query rewriting*. Această etapă încearcă să transforme o întrebare formulată natural într-o reprezentare mai utilă pentru căutare: o variantă corectată a întrebării, intenția probabilă, cuvinte-cheie și indicii de context, precum facultatea vizată. Rolul ei este să reducă efectul typo-urilor, al formulărilor incomplete și al sinonimelor, nu să stabilească adevărul factual al răspunsului. Survey-urile recente tratează această etapă ca parte a orchestration-ului RAG, alături de retrieval,

filtrare, generare și evaluare [GXG⁺23, GYZ⁺25].

Un flux RAG tipic conține colectarea documentelor, segmentarea lor în fragmente, indexarea embeddings-urilor, interpretarea întrebării, recuperarea fragmentelor relevante, filtrarea și generarea răspunsului pe baza contextului selectat [LPP⁺20, GXG⁺23]. Această schemă nu elimină complet riscurile: dacă etapa de retrieval selectează surse slabe sau dacă rescrierea întrebării schimbă sensul inițial, modelul poate formula un răspuns slab. De aceea, UVT Asist păstrează întrebarea originală, validează interpretarea lingvistică produsă de model și folosește mecanisme deterministe pentru selecția surselor: retrieval Qdrant, backfill lexical, reranking și scor de încredere calculat pe snapshot-ul local indexat.

2.3 Rulare locală, extensii browser și calitatea surselor

O extensie de browser este potrivită pentru aplicații care trebuie să fie disponibile rapid în timpul navigării. Chrome Manifest V3 definește structura extensiilor moderne, inclusiv fișierul manifest, popup-ul, permisiunile și accesul la API-uri de browser [Goo]. În cazul UVT Asist, extensia nu modifică paginile UVT și nu injectează logică în site-uri. Ea oferă un popup separat care comunică local cu backend-ul Flask.

O decizie importantă a proiectului este folosirea componentelor AI locale. Ollama este utilizat pentru interpretarea lingvistică a întrebării, nu pentru alegerea finală a surselor. În plus, Ollama este utilizat pentru generarea embeddings-urilor și, atunci când există dovezi suficiente, pentru formularea răspunsului final [Oll]. Această abordare reduce dependența de servicii externe și permite demonstrarea aplicației într-un mediu controlat. Rularea locală implică însă costuri: performanța depinde de hardware, modelele trebuie instalate pe mașina locală, iar Qdrant și Flask trebuie pornite înainte de utilizarea extensiei.

Într-un sistem conversațional obișnuit, calitatea răspunsului este evaluată în primul rând prin coerență și utilitate. Într-un sistem instituțional, aceste criterii nu sunt suficiente. Răspunsul trebuie să fie susținut de o sursă oficială, iar sursa trebuie să fie suficient de recentă pentru întrebarea adresată. În UVT Asist, paginile oficiale sunt tratate diferit în funcție de tip: paginile de navigare, documentele de metodologie, regulamentele și știrile datate primesc ponderi diferite în funcție de intenția întrebării.

UVT Asist aplică și principiul degradării controlate. Dacă Ollama nu poate interpreta lingvistic întrebarea, backend-ul păstrează întrebarea originală și continuă cu fallback raw. Dacă Qdrant sau embeddings-urile nu sunt disponibile, backend-

ul poate folosi fallback lexical pe indexul JSON. Dacă dovezile sunt prea slabe, generarea cu modelul de limbaj este evitată. Dacă indexarea de startup rulează, endpoint-ul de chat returnează un mesaj explicit cu progresul indexării. Pentru informații administrative, este mai acceptabil să spui „nu am suficiente dovezi oficiale” decât să produci un răspuns nesustținut.

Capitolul 3

Analiza problemei și cerințe

3.1 Ecosistemul UVT și riscurile informaționale

Ecosistemul web al UVT este format din mai multe surse oficiale: site-ul central, site-ul de admitere și site-urile facultăților. Din perspectiva unui sistem automat, această structură produce trei dificultăți. Prima este dispersia informației: unele răspunsuri sunt la nivelul universității, altele la nivelul facultății. A doua este eterogenitatea: paginile pot fi HTML, PDF, DOCX, text sau imagini care necesită OCR. A treia este dinamica informației: anumite pagini se schimbă periodic, în special cele legate de admitere, orare, burse sau calendare.

Un student nu formulează întrebări în termenii interni ai site-ului. El poate întreba „care e programul secretariatului?”, „unde văd orarul la info?”, „pot primi două burse?” sau „unde mă înscriu la admitere?”. Sistemul trebuie să transforme aceste formulări într-o căutare controlată, să aleagă domeniul potrivit și să returneze sursa oficială relevantă.

Riscul principal într-un asistent instituțional este producerea unei concluzii nejustificate. Un răspuns incorect despre burse, admitere sau proceduri poate avea consecințe mai mari decât o simplă recomandare greșită. Din acest motiv, sistemul trebuie să trateze informația administrativă ca informație cu sensibilitate ridicată. În proiect, această decizie se reflectă în reguli de prompt, în scorul de încredere și în alegerea de a nu apela modelul generativ atunci când nu există dovezi.

Un al doilea risc este alegerea unei surse oficiale, dar nepotrivite. O pagină a unei facultăți poate fi oficială, însă nu răspunde întrebării generale. O metodologie veche poate fi oficială, dar neapărat actuală. O știre despre admitere poate fi oficială, dar depășită. De aceea, proiectul nu folosește doar criteriul „domeniu oficial”, ci combină domeniul cu tipul paginii, titlul, URL-ul, conținutul și intenția interpretată și validată.

3.2 Cerințe funcționale și nefuncționale

Cerințele funcționale urmărite în proiect descriu modul în care utilizatorul interacționează cu sistemul și modul în care backend-ul procesează întrebările. Utilizatorul poate selecta facultatea relevantă sau poate lucra în contextul general UVT, apoi poate trimite întrebări în limbaj natural din popup-ul extensiei. Backend-ul transmite întrebarea către Ollama pentru interpretare lingvistică structurată și validează câmpurile `corrected_question`, `intent`, `keywords` și `faculty_hint`, astfel încât rezultatul modelului să fie tratat ca intrare controlată pentru căutare.

După interpretare, sistemul recuperează fragmente relevante din indexul local și le ordonează după scor, iar răspunsul returnat include surse oficiale distincte, afișate clar în interfață. Pentru întrebările ambigue, aplicația poate solicita alegerea unei facultăți, reducând riscul de a oferi un răspuns dintr-un context instituțional nepotrivit. De asemenea, aplicația expune starea componentelor prin endpoint-uri de health și status, iar utilizatorul poate oferi feedback asupra răspunsului primit.

Pe lângă funcționalități, proiectul impune cerințe nefuncționale importante. Sistemul trebuie să favorizeze sursele oficiale, să evite afirmațiile nesustținute, să funcționeze local, să degradeze controlat atunci când Qdrant sau Ollama nu sunt disponibile și să rămână utilizabil chiar dacă indexarea este în curs. O altă cerință este transparența: utilizatorul trebuie să vadă sursele, iar backend-ul trebuie să atașeze metadate despre încredere, modul de retrieval, modul de generare, profilul dovezilor, întrebarea originală și întrebarea interpretată.

Analiza proiectului arată că întrebările studenților pot fi împărțite în mai multe categorii: întrebări de navigare, întrebări administrative, întrebări de regulament, întrebări dependente de facultate și întrebări vagi sau incomplete. Această clasificare influențează arhitectura. Pentru întrebările de navigare, răspunsurile determinate sunt mai sigure și mai rapide decât generarea cu LLM. Pentru întrebările administrative, modelul poate fi util dacă există dovezi suficiente. Pentru întrebările de regulament, reranking-ul trebuie să favorizeze documentele de tip metodologie, procedură sau regulament. În toate cazurile, interpretarea produsă de Ollama este tratată ca intrare auxiliară pentru retrieval, nu ca decizie finală asupra sursei corecte.

Un caz de utilizare frecvent este căutarea orarului. Studentul nu are nevoie de o explicație lungă despre organizarea cursurilor, ci de pagina oficială pe care poate consulta orarul. Un al doilea caz este identificarea secretariatului sau a datelor de contact, unde răspunsul depinde adesea de facultatea selectată. Un caz mai dificil este întrebarea de tip regulament, de exemplu „Se pot cumula bursele?”, unde sistemul trebuie să favorizeze metodologii, regulamente sau documente instituționale.

3.3 Criterii de acceptare

Pentru ca prototipul să fie considerat funcțional, au fost stabilite câteva criterii de acceptare. Sistemul trebuie să pornească local, să expună starea componentelor, să încarce lista de facultăți și să răspundă la întrebări reprezentative. Pentru întrebările de orar și contact, sursa de top trebuie să fie pagina specifică facultății. Pentru întrebările de burse și regulamente, rezultatele trebuie să favorizeze documente sau pagini instituționale cu autoritate.

În plus, aplicația trebuie să ofere răspunsuri transparente. Fiecare răspuns trebuie să includă surse oficiale atunci când există dovezi, iar nivelul de încredere trebuie să fie vizibil. În cazul lipsei de dovezi, sistemul trebuie să returneze un mesaj de limitare. Pentru demonstrație, trebuie să fie prezentabile și stările de eșec: backend indisponibil, indexare în curs, Ollama indisponibil, Qdrant indisponibil și retrieval slab.

Capitolul 4

Arhitectura și implementarea sistemului

4.1 Arhitectura generală și structura proiectului

UVT Asist este construit ca sistem local, compus dintr-o extensie Chrome, un backend Flask, un index JSON, o bază vectorială Qdrant și modele Ollama pentru interpretare lingvistică, embeddings și generare [Goo, Pal, Qdr, Oll]. Produsul vizibil pentru utilizator este extensia, iar restul componentelor rulează local și susțin procesarea întrebării.

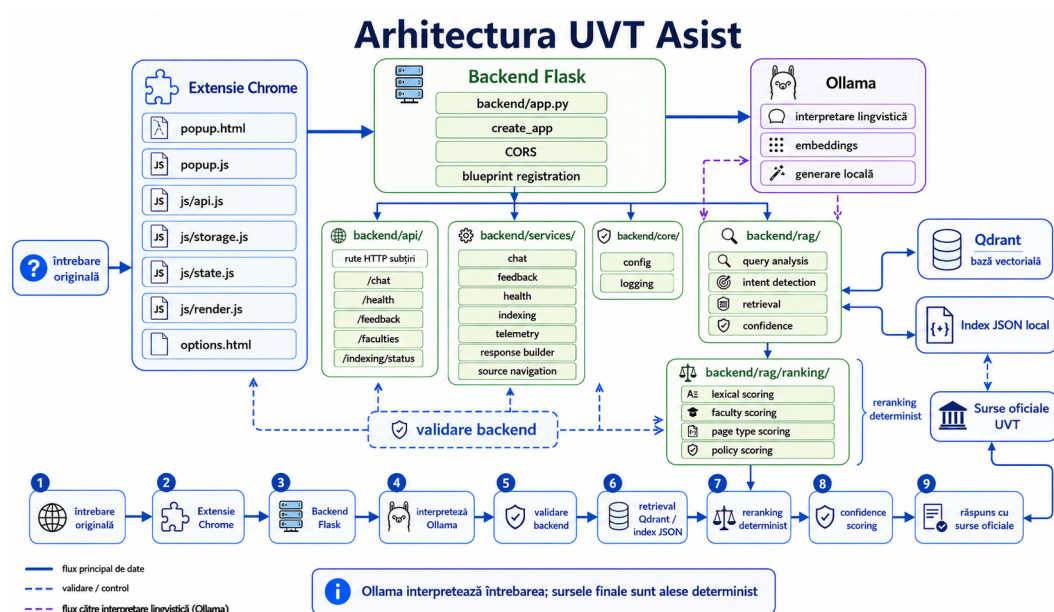


Figura 4.1: Arhitectura finală a sistemului UVT Asist

Separarea componentelor reduce complexitatea interfeței și permite dezvoltarea independentă a fiecărui nivel. Extensia nu trebuie să cunoască detaliile indexului sau ale modelului de limbaj. Backend-ul nu depinde de browser pentru logica de

retrieval. Qdrant poate fi înlocuit sau reconstruit fără modificarea popup-ului, iar modelul Ollama poate fi schimbat prin configurare.

Arhitectura a fost ghidată de câteva principii: separarea clară între interfață și procesarea informației, controlul asupra surselor, combinarea metodelor probabilistice cu reguli deterministe și rularea locală. Extensia Chrome trebuie să rămână simplă, predictibilă și ușor de folosit, iar responsabilitățile de validare JSON, safety guards, retrieval, scorare și generare aparțin backend-ului, unde pot fi testate mai riguros.

Proiectul aplicației este organizat în două zone principale: **backend/** și **extension/**, completate de documentație și scripturi de evaluare. În varianta curentă, backend-ul separă rutele HTTP, serviciile aplicației, configurația, logica RAG, ranking-ul, datele indexate și testele. Extensia Chrome este structurată similar: comunicarea cu backend-ul, stocarea locală, starea conversației, logica popup-ului și randarea interfeței sunt mutate în module JavaScript dedicate.

Componentă	Responsabilitate în structura curentă
backend/app.py	Punct de intrare Flask: definește <code>create_app</code> , configurează CORS și înregistrează blueprint-urile API.
backend/api/	Conține rute HTTP subțiri pentru chat, health, feedback, facultăți și indexare; rutele validează cererea și delegă logica către servicii.
backend/services/	Grupează serviciile de chat, feedback, health, indexare, telemetry, cache, navigare către surse, parsare cereri, safety guards, generare și construire a răspunsului.
backend/core/	Centralizează configurația aplicației și inițializarea logging-ului.
backend/rag/	Conține validarea interpretării produse de Ollama, păstrarea întrebării originale, fallback-ul raw, serviciul de retrieval și calculul nivelului de încredere.
backend/rag/ranking/	Separă scorarea lexicală, scorarea după facultate, scorarea după tipul paginii și scorarea pentru întrebări de politici sau regulamente.
backend/data/	Conține snapshot-ul local al indexului, folosit ca bază pentru fallback lexical și pentru reconstrucția colecției vectoriale.
backend/evaluation/	Conține seturile de evaluare versionate, inclusiv benchmark-ul independent de 1000 de întrebări și rapoartele asociate.
backend/scripts/	Conține utilitare pentru rularea evaluărilor, generarea rapoartelor și producerea graficelor de analiză.
backend/tests/	Verifică rutele, serviciile, indexarea, retrieval-ul, fallback-urile și cazurile de degradare controlată.
extension/popup.html, popup.css, popup.js	Definesc popup-ul extensiei, stilurile vizuale și inițializarea interacțiunilor din interfața principală.
extension/js/content.js	Conține logica de conținut și texte UI folosite de extensie, fără a muta retrieval-ul în browser.
extension/js/api.js	Gestionează comunicarea HTTP cu backend-ul local.
extension/js/storage.js	Gestionează <code>chrome.storage</code> , URL-ul backend-ului, istoricul conversației și întrebările recente.
extension/js/state.js	Păstrează starea conversației, facultatea selectată și tranzițiile UI relevante.
extension/js/render.js	Randează mesajele, sursele, nivelul de încredere, stările de eroare și indicatorii vizuali.
extension/options.html, options.js	Oferă interfața și logica de configurare pentru URL-ul backend-ului local.

Tabela 4.1: Structura principală a proiectului UVT Asist

4.2 Indexare, model de date și stocare vectorială

Unitatea principală a sistemului nu este pagina întreagă, ci fragmentul de pagină. Paginile web pot conține mult text irelevant, iar unele documente oficiale sunt lungi. Segmentarea în fragmente permite o căutare mai fină și reduce cantitatea de context trimisă modelului generativ, principiu folosit frecvent în sistemele de retrieval și RAG [MRS08, LPP⁺20]. Fiecare fragment este salvat atât în indexul JSON, cât și ca payload în Qdrant.

Câmp	Rol în sistem
chunk_id	Identificator stabil pentru fragment, folosit la deduplicare și la stocarea punctelor vectoriale.
faculty_id	Leagă fragmentul de contextul general UVT sau de o facultate concretă.
page_type	Clasifică pagina ca orar, contact, admitere, burse, regulamente, studenți sau general.
title	Oferă un semnal lexical puternic și este afișat în sursele returnate utilizatorului.
url	Păstrează legătura cu pagina oficială și permite deduplicarea surselor.
chunk_text	Conține textul efectiv folosit pentru retrieval, verificare și generare.
last_indexed	Indică momentul la care fragmentul a fost introdus în index.

Tabela 4.2: Câmpurile principale ale unui fragment indexat

Fluxul de indexare construiește baza de cunoștințe locală. Pentru fiecare sursă oficială configurată, sistemul pornește de la URL-uri de bază și de la rute prioritare, precum `/orare/`, `/burse/`, `/contact/`, `/admitere/`, `/regulamente/` sau `/metodologie/`. Dacă sunt disponibile sitemap-uri, acestea sunt folosite pentru descoperire mai largă. Apoi, link-urile candidate sunt filtrate astfel încât să rămână în domeniile oficiale permise.

Paginile acceptate sunt descărcate și convertite în text. Pentru HTML se elimină elementele irelevante precum meniuri, scripturi, formulare sau bannere. Pentru PDF și DOCX se extrage textul cu biblioteci dedicate, iar pentru imagini sau PDF-uri scanate există suport OCR opțional. Textul este limitat și împărțit în fragmente, pentru ca fiecare fragment să poată fi comparat eficient cu întrebările utilizatorilor.

Fiecare fragment primește metadata: identificator, facultate, tip de pagină, titlu, URL, text și momentul indexării. Documentul rezultat este salvat ca index JSON,

iar fragmentele sunt transformate în embeddings cu Ollama și încărcate în Qdrant [Oll, Qdr]. Qdrant stochează un punct vectorial pentru fiecare fragment și permite filtrarea după `faculty_id` și `page_type`, ceea ce reduce riscul ca o întrebare despre orar sau secretariat să primească surse de la altă facultate.

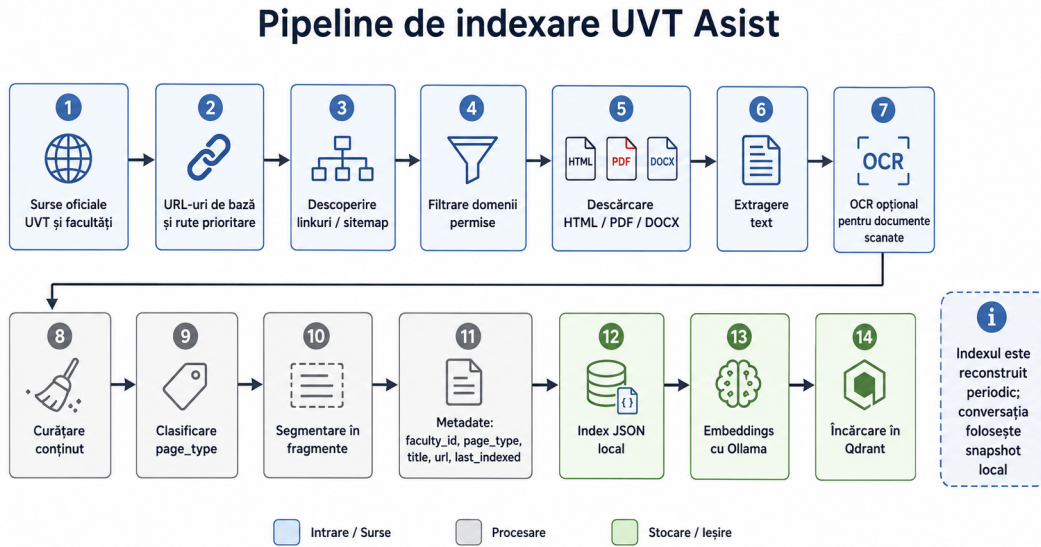


Figura 4.2: Pipeline-ul de indexare al surselor oficiale

4.3 Fluxul de interogare, retrieval, reranking și generare

La primirea unei întrebări, backend-ul păstrează textul original și aplică doar normalizare tehnică minimă: eliminarea spațiilor inutile, limitarea lungimii, validarea payload-ului JSON și verificări de siguranță. Corectarea semantică nu este implementată prin reguli locale în backend. În varianta curentă, backend-ul solicită de la Ollama un rezultat structurat cu patru câmpuri: `corrected_question`, `intent`, `keywords` și `faculty_hint`.

Rezultatul produs de Ollama este tratat ca ipoteză lingvistică, nu ca sursă de adevăr. Backend-ul verifică dacă răspunsul este JSON valid, dacă are câmpurile așteptate, dacă valorile sunt de tipul potrivit și dacă lungimea variantei corectate rămâne rezonabilă. Dacă Ollama este indisponibil sau întoarce JSON invalid, sistemul folosește întrebarea originală și fallback-ul raw. Dacă utilizatorul a selectat o facultate, selecția explicită are prioritate față de `faculty_hint`.

Întrebarea validată pentru retrieval este transformată în embedding cu Ollama și trimisă către Qdrant [Oll, Qdr]. Dacă embeddings-urile nu sunt disponibile, backend-ul poate folosi întrebarea originală în fallback lexical pe indexul

JSON. Căutarea vectorială folosește snapshot-ul local al surselor oficiale, cu filtre pe `faculty_id` și `page_type`. Pentru robustețe, sistemul execută mai multe treceri de căutare: întâi cu filtre stricte, apoi cu filtre mai largi. Rezultatele semantice sunt completate printr-un backfill lexical, util când vectorii nu surprind suficient semnalele explicite din URL sau titlu.

După recuperare, candidații sunt reordonați determinist. Scorul final combină suprapuneri lexicale, potrivirea facultății, preferința pentru tipul paginii, semnale din URL, penalizări pentru pagini generice și bonusuri pentru documente de regulament sau metodologie. Regulile din `backend/rag/ranking/` păstrează separată scorarea lexicală, scorarea pe facultate, scorarea pe tip de pagină și scorarea pentru întrebări de politici. Astfel, Ollama ajută la înțelegerea întrebării, dar alegerea finală a surselor rămâne în componentele de retrieval, reranking și confidence scoring.

Privit ca flux aplicativ, procesarea unei întrebări începe cu validarea cererii primite de la extensie. Backend-ul păstrează întrebarea originală, verifică structura payload-ului JSON, aplică normalizarea tehnică minimă și rulează verificările de siguranță. Acest pas este important deoarece întrebarea originală rămâne reperul la care sistemul poate reveni în cazul în care interpretarea lingvistică nu este disponibilă sau nu poate fi validată.

În continuare, backend-ul trimite către Ollama întrebarea și contextul minim necesar pentru interpretare. Rezultatul așteptat este un obiect structurat care conține varianta corectată a întrebării, intenția estimată, cuvintele-cheie și, dacă poate fi dedusă, o sugestie de facultate. Aceste informații sunt folosite ca semnale pentru retrieval, nu ca decizie finală. Ollama este utilizat pentru interpretarea lingvistică a întrebării, nu pentru alegerea finală a surselor. De aceea, backend-ul verifică schema, tipurile, lungimile și coerența rezultatului primit. Dacă răspunsul lipsește, este invalid sau nu respectă constrângerile definite, sistemul continuă cu întrebarea originală și cu fallback-ul raw.

După această etapă, sistemul stabilește facultatea efectivă folosită la căutare. Dacă utilizatorul a ales explicit o facultate în extensie, această alegere nu este suprascrisă de sugestia produsă de Ollama. Dacă întrebarea este dependentă de facultate, dar nu există suficient context pentru a o identifica, backend-ul nu inventează o direcție de căutare, ci returnează un mesaj de clarificare.

Căutarea surselor se face apoi în două moduri complementare. În cazul normal, întrebarea validată este transformată în embedding și sunt căutați candidați în Qdrant, cu filtre pentru facultate și tip de pagină. În paralel sau ca mecanism de completare, indexul JSON local este folosit pentru semnale lexicale explicite, mai ales în situațiile în care URL-ul, titlul paginii sau termenii administrativi sunt mai relevanți decât apropierea semantică. Dacă embeddings-urile nu sunt disponibile, sistemul poate funcționa în continuare prin căutare lexicală pe indexul JSON,

folosind întrebarea originală sau varianta validată.

Sursele candidate sunt apoi reordonate prin reguli deterministe. Reranking-ul combină potrivirea lexicală, potrivirea facultății, tipul paginii, semnalele din URL și penalizările pentru pagini prea generale. Pe baza acestor scoruri se calculează nivelul de încredere. Pentru întrebările de navigare, atunci când există o sursă suficient de clară, răspunsul poate fi construit direct. Dacă dovezile sunt slabe, sistemul preferă un mesaj de limitare în locul unui răspuns nesigur. Doar când sursele selectate oferă suport suficient, backend-ul construiește promptul RAG și folosește Ollama pentru formularea răspunsului final. Rezultatul trimis extensiei conține răspunsul, sursele oficiale și metadatele de încredere.

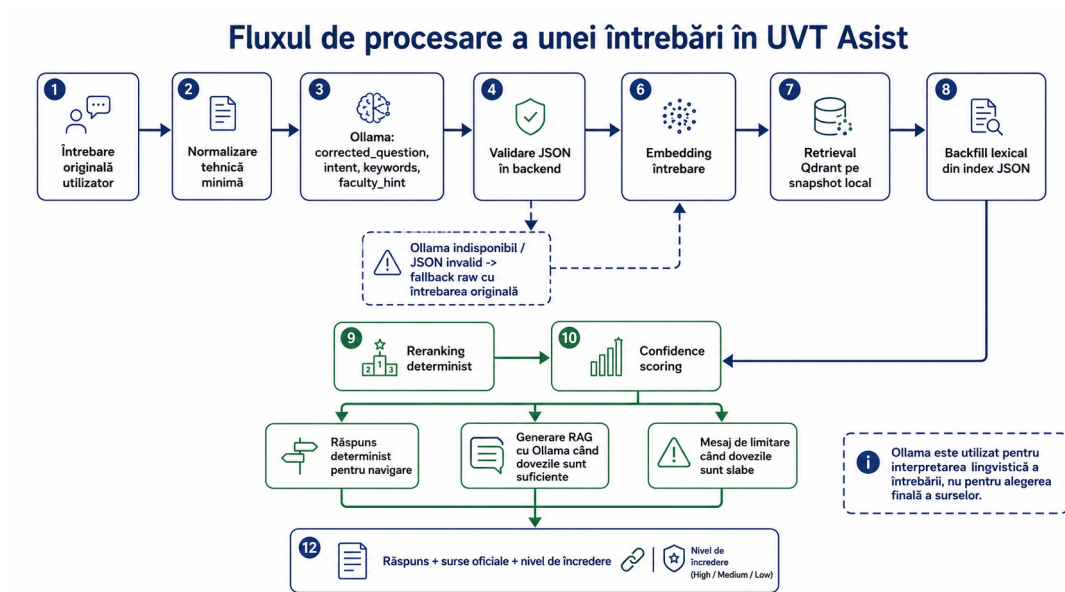


Figura 4.3: Pipeline-ul de procesare a unei întrebări în UVT Asist

Generarea cu modelul de limbaj este folosită doar când există dovezi suficiente. Pentru întrebări de navigare, precum orar, contact sau admitere, sistemul poate construi răspunsul direct, fără LLM. Pentru întrebări cu încredere scăzută, backend-ul returnează un mesaj de limitare și sursele apropiate, evitând generarea unui text care ar putea părea mai sigur decât este. Scorul de încredere ține cont de puterea rezultatului principal, diferența față de următorul rezultat, numărul de surse distincte și semnalele de potrivire. Ollama nu este tratat ca sursă a adevărului, ci ca un modul local pentru interpretare lingvistică și, în cazurile permise, formularea răspunsului pe baza surselor deja selectate.

Capitolul 5

Interfață și funcționalități

5.1 Extensia Chrome și experiența de utilizare

Extensia Chrome este interfața principală a aplicației și respectă modelul de organizare descris în documentația Manifest V3 [Goo]. Ea oferă un popup cu selector de facultate, câmp pentru întrebare, afișare de mesaje, surse oficiale, nivel de încredere, indicatori pentru starea indexului local și stare a backend-ului. Extensia păstrează istoricul conversației pe facultate și o listă scurtă de întrebări recente folosind mecanismul de stocare locală al browserului.

Din punct de vedere al produsului, această interfață este potrivită pentru scenariul de utilizare. Studentul poate întreba rapid fără a naviga manual prin mai multe meniuri. Din punct de vedere tehnic, extensia trimite cereri HTTP către backend-ul local prin `extension/js/api.js`, păstrează URL-ul backend-ului și istoricul prin `extension/js/storage.js`, gestionează starea conversației prin `extension/js/state.js` și randează interfața prin `extension/js/render.js`.

Popup-ul nu încearcă să ascundă starea tehnică a sistemului. Dacă backend-ul este disponibil și indexul este pregătit, utilizatorul vede o stare de sistem pregătit. Dacă indexarea rulează, interfața afișează progresul. Dacă backend-ul nu răspunde, butoanele rămân controlate, iar mesajul indică faptul că serviciul local trebuie pornit. Pagina `extension/options.html` permite configurarea URL-ului backend-ului local fără modificarea codului.

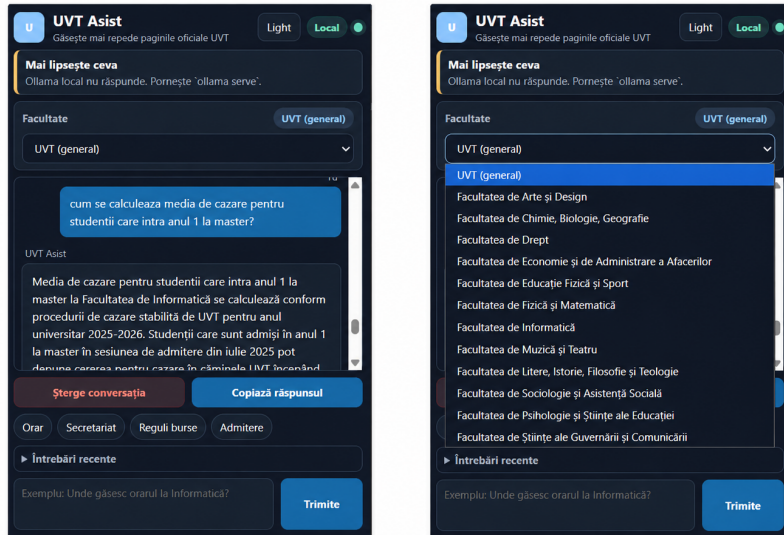


Figura 5.1: Popup-ul extensiei UVT Asist și selectorul de facultate

5.2 Backend, endpointuri, configurare și observabilitate

Backend-ul Flask este componenta de orchestrare. Flask este un framework web Python minimalist, potrivit pentru prototipuri și servicii API locale [Pal]. În proiect, `backend/app.py` definește `create_app`, configurează CORS și înregistrează blueprint-urile din `backend/api/`. Rutele HTTP rămân subțiri și delegă logica către serviciile din `backend/services/`.

Endpoint-ul `/health` raportează disponibilitatea Ollama, modelele configurate, starea indexului JSON, starea colecției Qdrant, modul de retrieval, cache-urile și progresul indexării. Endpoint-ul `/indexing/status` permite extensiei să afișeze progresul indexării. Endpoint-ul `/feedback` salvează feedback-ul utilizatorului într-un fișier JSONL, util pentru analiză și îmbunătățiri ulterioare.

Endpoint	Rol
/health	Returnează starea Ollama, Qdrant, index JSON, cache-uri, mod de retrieval și readiness.
/faculties	Returnează lista facultăților configurate pentru selectorul din extensie.
/indexing/status	Returnează progresul indexării de startup.
/chat	Primește întrebarea, facultatea și istoricul, apoi returnează răspunsul și sursele.
/feedback	Salvează votul utilizatorului și metadatele răspunsului pentru analiză ulterioară.

Tabela 5.1: Endpoint-urile principale ale backend-ului

Configurarea aplicației și logging-ul sunt centralizate în `backend/core/`. Serviciile de chat, feedback, health, indexare și telemetry sunt grupate în `backend/services/`. Backend-ul aplică validări defensive: întrebările goale primesc răspuns dedicat, payload-urile invalide nu produc eroare internă, istoricul este limitat ca număr de mesaje și lungime, iar întrebarea este trunchiată la o dimensiune configurabilă. În plus, rezultatul structurat primit de la Ollama este verificat înainte de utilizare; dacă schema sau tipurile nu sunt valide, fluxul continuă cu întrebarea originală.

5.3 Răspunsuri, surse, feedback și degradare controlată

Răspunsul trimis către extensie conține textul final, lista surselor, facultatea potrivită, scorul de încredere, profilul întrebării, modul de retrieval, modul de generare și profilul dovezilor. Metadatele pot păstra atât întrebarea originală, cât și varianta interpretată folosită pentru retrieval, pentru a permite auditarea diferenței dintre formularea utilizatorului și forma procesată. Sursele sunt deduplicate pe URL și afișate ca link-uri oficiale. Astfel, utilizatorul poate verifica imediat pagina pe care se bazează răspunsul.

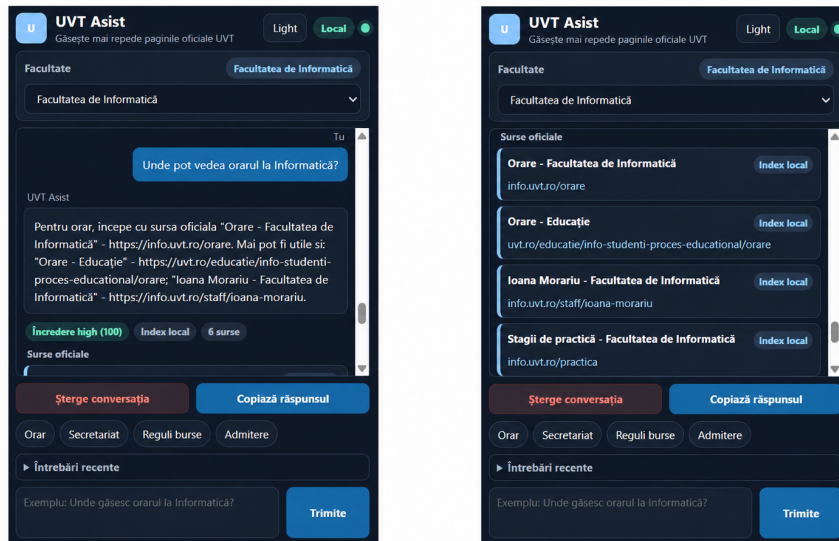


Figura 5.2: Afișarea răspunsului, surselor oficiale și nivelului de încredere

Feedback-ul utilizatorului este salvat împreună cu întrebarea originală, răspunsul, sursele, scorul de încredere și modul de generare. Într-o etapă ulterioară, aceste date pot fi folosite pentru identificarea întrebărilor problematice, a surselor slab indexate, a cazurilor în care interpretarea lingvistică a schimbat prea mult sensul sau a situațiilor în care reranking-ul trebuie ajustat.

Indexul local oferă viteză și stabilitate, dar paginile oficiale se pot schimba. În varianta curentă, aplicația nu redescarcă paginile oficiale la fiecare întrebare; prospețimea informației este gestionată prin reconstruirea indexului JSON și a colecției Qdrant. Această alegere face răspunsurile mai reproductibile în evaluare și limitează dependența de disponibilitatea site-urilor în momentul conversației, cu prețul unei responsabilități clare de reindexare periodică.

Aplicația degradează controlat atunci când unele servicii lipsesc temporar. Dacă Ollama nu este disponibil pentru interpretarea lingvistică, backend-ul păstrează întrebarea originală și continuă prin fallback raw. Dacă Qdrant sau embeddings-urile nu sunt disponibile, backend-ul poate folosi fallback lexical pe indexul JSON. Dacă dovezile sunt prea slabe, generarea cu modelul de limbaj este evitată. Dacă indexarea de startup rulează, endpoint-ul de chat returnează un mesaj explicit cu progresul indexării.

Capitolul 6

Validare și evaluare

6.1 Testare automată și scenarii manuale

Validarea prototipului se face prin teste automate și prin scenarii manuale de demonstrație. Testele automate acoperă componente esențiale: interpretarea întrebărilor cu typo-uri, preferința pentru pagini stabile de admitere față de știri vechi, clasificarea paginilor, normalizarea URL-urilor, rezistența la payload-uri greșite și comportamentul sistemului când indexarea este în desfășurare.

O parte dintre teste verifică interpretarea lingvistică a întrebării și efectul ei asupra retriever-ului. Pentru typo-uri, de exemplu formularea „orarul”, sistemul trebuie să poată obține o intenție de orar fără să piardă întrebarea originală. Pentru întrebările ambigue, testele verifică faptul că backend-ul cere clarificare sau respectă facultatea selectată explicit, în loc să accepte necritic un `faculty_hint`. Pentru admitere, testul confirmă că o pagină stabilă despre procesul de admitere este preferată față de o știre veche. O altă categorie de teste verifică indexul: normalizarea URL-urilor, detectarea tipului de pagină, existența câmpurilor necesare pentru payload-ul vectorial și limitarea dimensiunii fragmentelor.

Testele backend verifică robustețea API-ului. Un payload JSON greșit nu trebuie să producă o eroare 500. Un răspuns JSON invalid primit de la Ollama trebuie respins, iar sistemul trebuie să continue cu întrebarea originală. Când Ollama este indisponibil, fluxul trebuie să folosească fallback raw fără corectare semantică. O întrebare cu dovezi slabe nu trebuie să invoce modelul de limbaj pentru generare. O întrebare de navigare cu rezultat sigur trebuie să primească răspuns determinist. De asemenea, când indexarea este în desfășurare, chat-ul trebuie să se oprească temporar și să returneze statusul de indexare.

Scenariile manuale verifică situații reprezentative pentru studenți: întrebarea despre orar trebuie să favorizeze pagina oficială de orare, întrebarea despre secretariat trebuie să favorizeze pagina de contact, întrebările despre cumularea burselor

trebuie să favorizeze metodologii sau regulamente oficiale, iar oprirea backend-ului trebuie să fie afișată clar în extensie. Evaluarea manuală urmărește sursa de top, calitatea răspunsului și metadatele afișate.

6.2 Evaluare pe seturi de întrebări

Pentru ca rezultatele să poată fi interpretate și refăcute în aceleași condiții, Tabelul 6.1 sintetizează principalele artefacte și setări folosite în evaluările raportate. Valorile provin din fișierele versionate ale proiectului aplicației, din artefactele disponibile în mediul local de dezvoltare și din documentația de evaluare; acolo unde o informație nu este consemnată explicit, acest lucru este indicat în tabel.

Câmp	Valoare identificată
Versiune proiect evaluat	Commit Git evaluat: <code>e2dbddb</code> ; comenzile de rulare și detaliile de configurare sunt documentate în <code>README.md</code> .
Perioada evaluărilor	Q&A 100: 2026-06-25; Q&A 1000: 2026-06-25T13:16:19Z-2026-06-26T16:12:08Z.
Seturi de întrebări	Q&A 100: <code>backend/evaluation/eval_qa_100.json</code> ; Q&A 1000: <code>backend/evaluation/eval_qa_1000_independent.json</code> .
Snapshot index local	<code>backend/data/page_index.json</code> , construit la 2026-06-02T01:41:32+00:00.
Dimensiune index	30 707 pagini și 349 835 fragmente, conform <code>backend/data/page_index.meta.json</code> .
Qdrant	Colecție configurată: <code>uv_t_asist_chunks</code> ; imagine Docker: <code>qdrant/qdrant:v1.14.1</code> ; dimensiune vectori: 768; metrică: Cosine, conform <code>qdrant_storage/collections/uv_t_asist_chunks/config.json</code> .
Modele Ollama	Generare și interpretare: <code>qwen3:4b</code> ; embeddings: <code>omic-embed-text</code> ; query analysis activat prin <code>OLLAMA_QUERY_ANALYSIS_ENABLED=true</code> .
Versiuni Python/backend	Python runtime local: 3.11.9, conform <code>.venv/pyvenv.cfg</code> ; <code>Flask==3.0.3</code> , <code>flask-cors==4.0.1</code> , <code>qdrant-client>=1.14,<2</code> , conform <code>backend/requirements.txt</code> .
Scripturi de evaluare	Q&A 100: <code>backend/scripts/evaluate_qa.py</code> ; Q&A 1000: <code>backend/scripts/evaluate_qa_1000_independent.py</code> cu <code>-run-label final_1000 -resume</code> .
Mediu hardware	Laptop local cu Windows 11 Pro 64-bit, CPU 12th Gen Intel(R) Core(TM) i7-12700H cu 14 nuclee și 20 fire de execuție, 16 GB DDR4 3200 MT/s RAM, GPU dedicat NVIDIA GeForce RTX 3050 Laptop GPU cu 4 GB memorie dedicată, GPU integrat Intel Iris Xe Graphics și SSD NVMe Micron_2400_MTFDKBA512QFM. Latențele trebuie interpretate ca rezultate ale acestei configurații locale.

Tabela 6.1: Condiții de reproductibilitate pentru evaluările raportate

Prima evaluare cantitativă folosită în proiect a fost setul Q&A de 100 de întrebări definit intern. Fiecare întrebare a fost trimisă către endpoint-ul de chat, iar evaluatorul a verificat scorul Q&A, sursele returnate, potrivirea nivelului de încredere, comportamentul pentru întrebări fără răspuns sigur și latența. Raportarea compară rezultatele înainte și după optimizările aplicate asupra selecției surselor, reranking-ului, tratării întrebărilor vagi și controlului generării.

Rezultate înainte și după optimizări pe setul Q&A 100

Metrică	Înainte	După	Diferență
 Întrebări evaluate	100	100	0
 Rată de trecere	65%	100%	↑ +35 pp
 Scor mediu Q&A	69.54	85.84	↑ +16.30
 Scor median Q&A	82.0	85.0	↑ +3.0
 Top-1 URL corect	98/100	100/100	↑ +2
 Top-3 URL corect	98/100	100/100	↑ +2
 Potrivire nivel de încredere	83/100	99/100	↑ +16
 Întrebări fără răspuns sigur tratate corect	1/10	10/10	↑ +9
 Latență medie	27.182s	13.679s	↓ -13.503s
 Latență mediană	37.689s	4.342s	↓ -33.347s

Rezultate valabile pe setul definit de întrebări și configurația locală evaluată.

Figura 6.1: Rezultatele evaluării înainte și după optimizări

Rezultatele indică o îmbunătățire consistentă pe setul definit de întrebări, în special pentru întrebările fără răspuns sigur și pentru potrivirea nivelului de încredere. Creșterea Top-1 și Top-3 URL la 100/100 arată că, în acest set, sursele oficiale așteptate au fost plasate corect în lista de rezultate. Totuși, aceste rezultate nu reprezintă o garanție generală pentru orice întrebare posibilă. Ele descriu comportamentul sistemului pe datasetul versionat în proiect, cu indexul, configurația și serviciile locale folosite la momentul evaluării.

Îmbunătățirile provin din trei direcții principale: selecția mai deterministă a surselor oficiale, refuzul sau clarificarea în cazurile nesigure și optimizările care reduc latența pentru întrebările frecvente. Latența medie și latența mediană descriu aspecte diferite ale comportamentului sistemului. Media este sensibilă la câteva cazuri lente, în timp ce mediana descrie mai bine timpul tipic observat pentru o întrebare obișnuită din setul evaluat.

Pentru o verificare mai largă, aplicația a fost evaluată ulterior pe un benchmark independent de 1000 de întrebări. În această evaluare, `ideal_answer` este folosit ca rubrică de evaluare, nu ca text care trebuie reprodus exact. Evaluatorul automat analizează răspunsul și metadatele returnate de sistem: sursele, intenția, `page_`

`type`, nivelul de încredere, termenii obligatorii din `required_terms` și termenii interziși din `forbidden_terms`. Întrebările fără răspuns sigur sunt evaluate separat, deoarece un comportament corect poate însemna refuzul de a formula o afirmație factuală nesusținută.

Indicator global	Valoare
Total întrebări	1000
Răspunsuri trecute	644
Răspunsuri eșuate	356
Rata de trecere	64.4%
Scor mediu	72.34
Scor median	82.35
Top-1 URL match	$439/694 = 63.26\%$
Top-3 URL match	$502/694 = 72.33\%$
Potrivire nivel de încredere	$843/1000 = 84.30\%$
Erori tehnice	0

Tabela 6.2: Rezultate globale pe benchmark-ul independent de 1000 de întrebări

Categorie	Rată de trecere
admitere	80.0%
burse	65.0%
calendar_academic	39.0%
cazare_camino	87.0%
contact_secretariat	92.0%
intrebări_fără_răspuns_sigur	43.0%
intrebări_vagi_ambigue	16.0%
orar	68.0%
regulamente_metodologii	59.0%
voluntariat_credite	95.0%

Tabela 6.3: Rezultate pe categorii în benchmark-ul independent

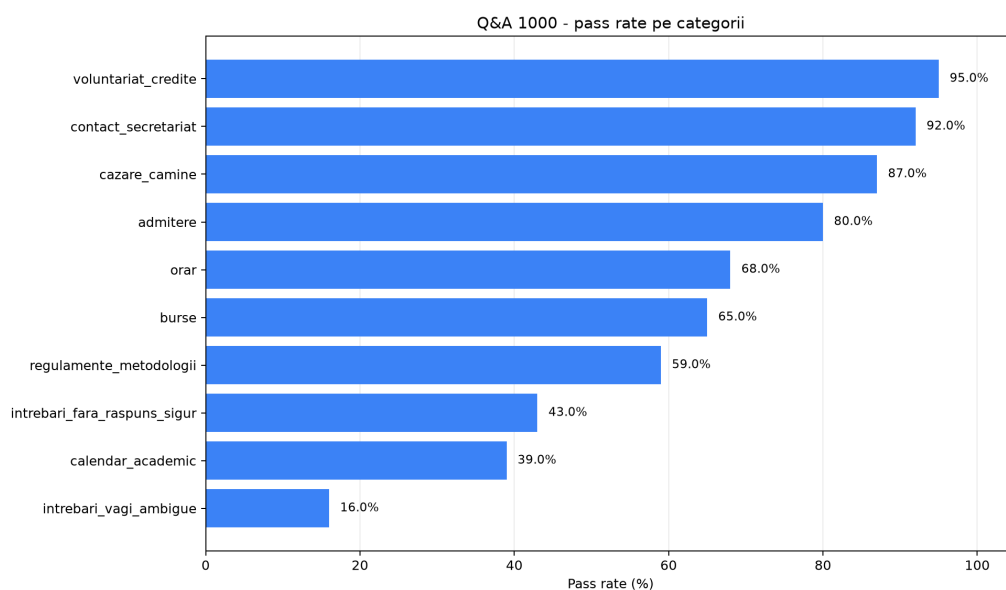


Figura 6.2: Rata de trecere pe categorii în benchmark-ul independent Q&A 1000

Indicator de latență	Valoare
Latență medie	23.323 s
Latență mediană	24.335 s
P90	39.986 s
P95	51.269 s

Tabela 6.4: Latența observată în benchmark-ul independent

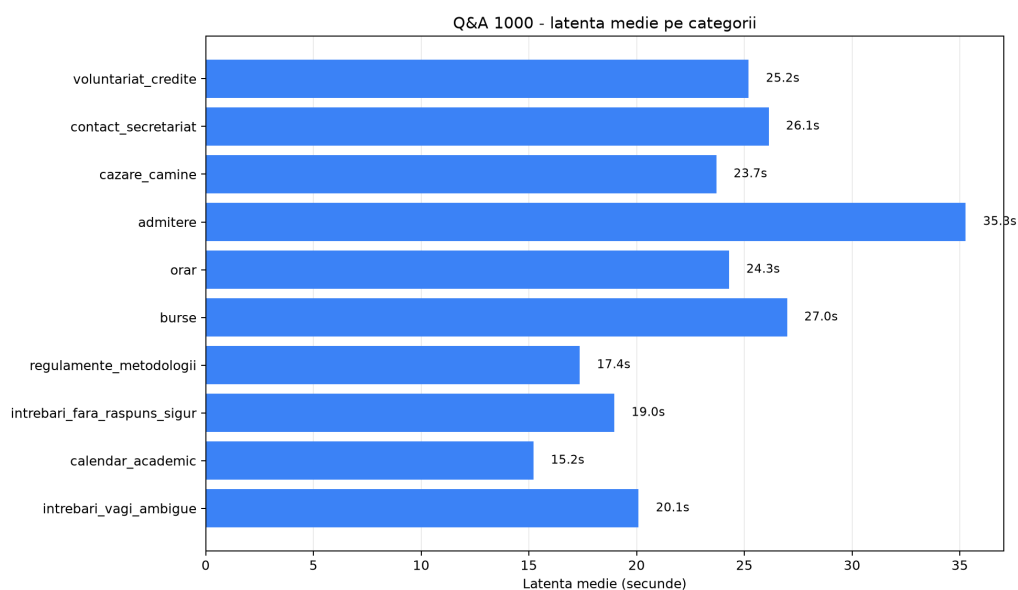


Figura 6.3: Latența medie pe categorii în benchmark-ul independent Q&A 1000

Aplicația a generat răspuns pentru toate cele 1000 de întrebări și nu au existat erori tehnice raportate. Rezultatele sunt însă neuniforme pe categorii. Categoriile forte sunt `voluntariat_credite`, `contact_secretariat`, `cazare_camine` și `admitere`, unde informația este mai bine delimitată sau poate fi asociată mai clar cu surse oficiale. Categoriile slabe sunt `intrebări_vagi_ambigue`, `calendar_academic` și `intrebări_fara_raspuns_sigur`; aceste rezultate indică dificultăți în clarificarea întrebărilor incomplete, în identificarea calendarului potrivit și în refuzarea sigură a întrebărilor pentru care nu există dovezi suficiente.

Latența arată limitele rulării locale. Întrebările pot deveni lente atunci când activează pași suplimentari: interpretarea lingvistică prin Ollama, calculul embeddings-urilor locale, interogări Qdrant cu mai multe treceri, fallback lexical, construcția promptului RAG sau generarea răspunsului cu Ollama. Valorile P90 și P95 arată că o parte dintre întrebări depășesc semnificativ latența mediană, ceea ce este relevant pentru experiența utilizatorului.

Rapoartele tehnice ale benchmark-ului arată și modul în care sistemul a produs răspunsurile. Modul de generare `ollama` apare în 458 de cazuri, `local_source_navigation` în 347 de cazuri, `fallback_ollama_error` în 105 cazuri, `none` în 61 de cazuri și `clarification` în 29 de cazuri. Distribuția backend-ului de retrieval arată 910 cazuri cu Qdrant, 58 de cazuri de clarificare și 32 de cazuri oprite de guard-uri pentru întrebări nesuținute. Aceste valori confirmă că aplicația nu se comportă ca un chatbot generic: o parte importantă din răspunsuri sunt de navigare locală sau de limitare, iar Qdrant rămâne mecanismul principal de recuperare a surselor.

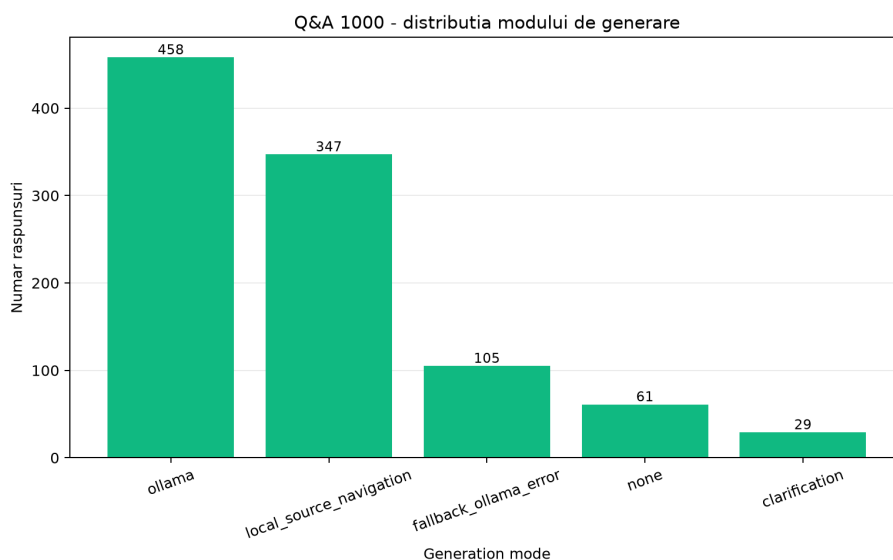


Figura 6.4: Distribuția modului de generare în benchmark-ul independent Q&A 1000

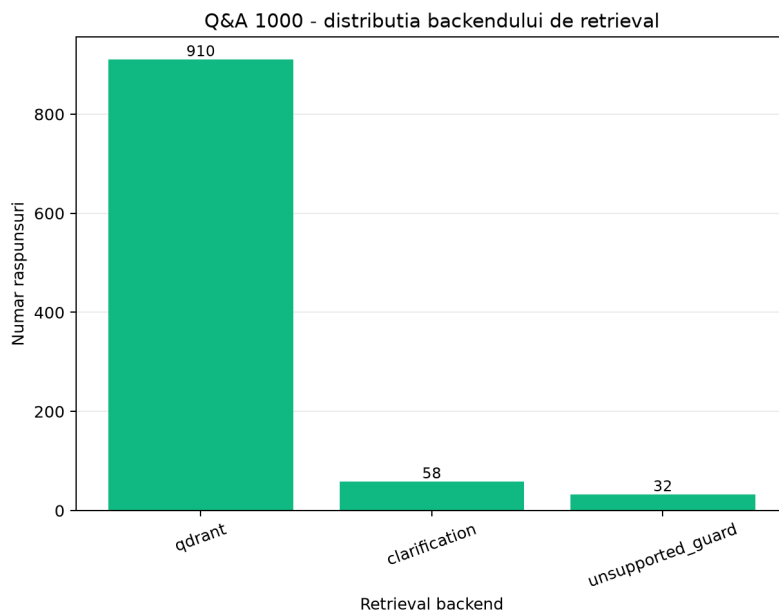


Figura 6.5: Distribuția backend-ului de retrieval în benchmark-ul independent Q&A 1000

Rezultatele trebuie interpretate prudent. Ele sunt valabile pentru setul definit de 1000 de întrebări și pentru configurația locală folosită la momentul evaluării. Nu reprezintă o garanție universală pentru orice întrebare posibilă, iar evaluatorul automat nu înlocuiește evaluarea umană. Rolul lui este să ofere o măsurare repetabilă și mai largă decât scenariile manuale, nu să epuizeze analiza calității răspunsurilor.

6.3 Limite, amenințări la validitate, securitate și confidențialitate

Sistemul depinde de calitatea indexului local. Dacă o pagină oficială lipsește din index sau dacă textul nu poate fi extras corect, răspunsul poate fi incomplet. Documentele PDF scanate pot necesita OCR, iar calitatea OCR-ului poate varia. De asemenea, dacă o pagină se modifică după indexare, sistemul poate folosi informații depășite până la reconstrucția indexului. Această limită trebuie tratată prin reindexare periodică și prin monitorizarea surselor oficiale importante.

O altă limită este performanța locală. Ollama și Qdrant trebuie să ruleze pe mașina utilizatorului sau pe o mașină de demonstrație. Timpul de răspuns depinde de modelul ales, de dimensiunea indexului și de hardware. Pentru o utilizare instituțională reală, ar putea fi necesară migrarea către un serviciu administrat intern, cu resurse controlate.

Mutarea corectării semantice către Ollama introduce și riscul de *meaning drift*:

modelul poate rescrie o întrebare astfel încât varianta corectată să fie fluentă, dar ușor diferită de intenția utilizatorului. De exemplu, o întrebare vagă despre „taxe” poate fi interpretată ca taxă de școlarizare, taxă de admitere sau taxă administrativă. Pentru a limita acest risc, backend-ul păstrează întrebarea originală, validează schema și lungimea câmpurilor returnate de Ollama, respectă selecția explicită a facultății și poate reveni la fallback raw atunci când interpretarea este invalidă sau nesigură. Această validare reduce riscul operațional, dar nu transformă interpretarea produsă de Ollama într-o sursă de adevăr.

O amenințare la validitate este dimensiunea setului de întrebări folosit pentru testare. Scenariile selectate acoperă întrebări importante, dar nu includ toate formulările posibile ale studenților. O a doua amenințare este dependența de starea site-urilor oficiale în momentul indexării. O a treia amenințare este variația modelelor locale: schimbarea modelului de interpretare, a modelului de generare sau a modelului de embedding poate modifica atât analiza întrebării, cât și scorurile semantice.

Prototipul rulează local și nu trimite întrebările către un serviciu AI extern. Acest lucru este un avantaj pentru confidențialitate, dar nu elimină toate responsabilitățile. Backend-ul acceptă cereri HTTP locale, deci trebuie tratat ca un serviciu de dezvoltare, nu ca un server expus public. Feedback-ul este salvat local în format JSONL și poate conține întrebări formulate liber de utilizatori, motiv pentru care o versiune distribuită ar trebui să includă reguli de retenție, anonimizare și informare.

Capitolul 7

Concluzii și direcții viitoare de dezvoltare

Lucrarea a prezentat proiectarea unui asistent inteligent local pentru accesul la informațiile oficiale UVT. Problema abordată este una practică: informația există, dar este distribuită între mai multe site-uri și nu este întotdeauna ușor de găsit rapid. UVT Asist propune o interfață conversațională integrată în browser, susținută de un backend care caută în indexul local, selectează surse oficiale și sintetizează răspunsuri pe baza acestora.

Din punct de vedere teoretic, soluția se bazează pe combinarea regăsirii informației cu modelele de limbaj. Căutarea semantică oferă flexibilitate, interpretarea lingvistică locală ajută la tratarea typo-urilor și a întrebărilor incomplete, iar regulile deterministe oferă control asupra selecției surselor. Prin afișarea surselor, scorului de încredere și metadatelor despre modul de retrieval, sistemul urmărește să rămână transparent și prudent.

Prototipul demonstrează că o arhitectură RAG locală este fezabilă pentru un scenariu universitar. Totuși, calitatea finală depinde de acoperirea indexului, de actualizarea surselor și de evaluarea continuă pe întrebări reale. Rezultatele trebuie citite ca verificare pe seturile de evaluare definite în proiect, nu ca garanție universală pentru orice întrebare posibilă.

Contribuția principală constă în construirea unei soluții aplicate pentru ecosistemul UVT, cu extensie Chrome, backend Flask modular, index local, Qdrant, Ollama pentru interpretare și generare controlată, plus mecanisme de reranking determinist. Structura actuală separă rutele HTTP, serviciile, configurația, analiza RAG și ranking-ul, ceea ce face aplicația mai ușor de testat și de extins.

La nivel funcțional, sistemul oferă selecția facultății, istoric conversațional, întrebări recente, afișarea surselor oficiale, nivel de încredere, indicatori pentru indexul local și feedback. La nivel de siguranță, aplicația poate refuza sau limita răspunsul

atunci când dovezile oficiale sunt slabe, evitând formularea unor afirmații speculative.

Pe setul Q&A de 100 de întrebări definit în proiect, rata de trecere a crescut de la 65% la 100%, scorul mediu Q&A de la 69.54 la 85.84, iar scorul median de la 82.0 la 85.0. Top-1 URL corect și Top-3 URL corect au ajuns la 100/100, potrivirea nivelului de încredere la 99/100, iar întrebările fără răspuns sigur tratate corect de la 1/10 la 10/10. Latența medie a scăzut de la 27.182 s la 13.679 s, iar latența mediană de la 37.689 s la 4.342 s.

Evaluarea independentă pe 1000 de întrebări a oferit o imagine mai strictă asupra comportamentului sistemului. Aplicația a generat răspuns pentru toate întrebările și nu a produs erori tehnice, însă rata de trecere a fost 64.4%, cu scor mediu 72.34 și scor median 82.35. Rezultatele cele mai bune au fost în categoriile `voluntariat_credite`, `contact_secretariat`, `cazare_camine` și `admitere`, iar rezultatele cele mai slabe în `intrebări_vagi_ambigue`, `calendar_academic` și `intrebări_fără_răspuns_sigur`. Aceste valori arată că sistemul este funcțional, dar încă sensibil la întrebări ambigue și la situații în care răspunsul sigur trebuie delimitat mai strict.

Direcțiile de dezvoltare includ reindexare programată, monitorizarea modificărilor în sursele oficiale, extinderea setului de teste, evaluare cu studenți reali și îmbunătățirea OCR-ului pentru documente scanate. Pe baza benchmark-ului independent, prioritățile tehnice sunt tratarea întrebărilor vagi, îmbunătățirea acoperirii pentru calendarul academic și refuzul mai precis al întrebărilor fără răspuns sigur. O altă direcție utilă este construirea unui set standard de întrebări și răspunsuri de referință, pentru a compara obiectiv variante diferite de interpretare lingvistică, embedding, reranking și modele generative.

La nivel de interfață, extensia poate fi extinsă cu explicații mai clare pentru întrebările neacoperite, gruparea surselor pe categorii și sugestii de reformulare. La nivel de backend, ar fi utilă separarea mai explicită între răspunsuri de navigare, răspunsuri administrative și răspunsuri de regulament, astfel încât fiecare categorie să aibă reguli de verificare adaptate.

Bibliografie

- [Goo] Google Chrome Developers. Chrome extensions: Manifest v3 documentation. <https://developer.chrome.com/docs/extensions/>. Accesat: 26 mai 2026.
- [GXG⁺23] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- [GYZ⁺25] Aoran Gan, Hao Yu, Kai Zhang, Qi Liu, Wenyu Yan, Zhenya Huang, Shiwei Tong, and Guoping Hu. Retrieval augmented generation evaluation in the era of large language models: A comprehensive survey. *arXiv preprint arXiv:2504.14891*, 2025.
- [JDJ17] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *arXiv preprint arXiv:1702.08734*, 2017.
- [KOM⁺20] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781, 2020.
- [LPP⁺20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [MRS08] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schutze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

- [Oll] Ollama. Ollama documentation. <https://github.com/ollama/ollama/tree/main/docs>. Accesat: 26 mai 2026.
- [Pal] Pallets Projects. Flask documentation. <https://flask.palletsprojects.com/>. Accesat: 26 mai 2026.
- [Qdr] Qdrant. Qdrant documentation. <https://qdrant.tech/documentation/>. Accesat: 26 mai 2026.
- [RG19] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992, 2019.
- [RZ09] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [VSP⁺17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Anexa A

Utilizarea instrumentelor de inteligență artificială generativă

În această anexă este prezentată o sinteză a modului în care au fost utilizate instrumentele de inteligență artificială generativă în cadrul lucrării și al proiectului software asociat. Instrumentele AI au fost folosite ca sprijin auxiliar pentru organizare, verificări de coerență și sugestii tehnice. Codul, structura finală, interpretarea rezultatelor și textul final au fost revizuite și asumate de autor.

Instrument AI	Scopul utilizării	Zone în care a fost utilizat	Observații
OpenAI Codex / ChatGPT	Sprijin pentru organizarea textului, sugestii de refactorizare, formulări de documentație și verificări de coerență	Lucrarea LaTeX, documentația proiectului și unele decizii de structurare a codului	Instrumentul nu este tratat ca sursă bibliografică și nu este autor al lucrării; rezultatele au fost revizuite de autor.

Tabela A.1: Sinteza utilizării instrumentelor de inteligență artificială generativă

A.1 Detalii privind utilizarea OpenAI Codex / ChatGPT

Instrumentul a fost utilizat pentru sprijin punctual în organizarea materialului, pentru reformulări de documentație și pentru verificarea coerenței dintre textul lucrării și structura curentă a aplicației. În zona de cod, sugestiile au vizat în principal refactorizarea și separarea responsabilităților, nu înlocuirea procesului de proiectare sau validare.

Autorul a verificat manual modificările, a păstrat controlul asupra conținutului final și a interpretat rezultatele evaluării pe baza rapoartelor existente în proiect. Afirmările cantitative din lucrare provin din seturile de evaluare definite în repository, nu din estimări generate de AI.